

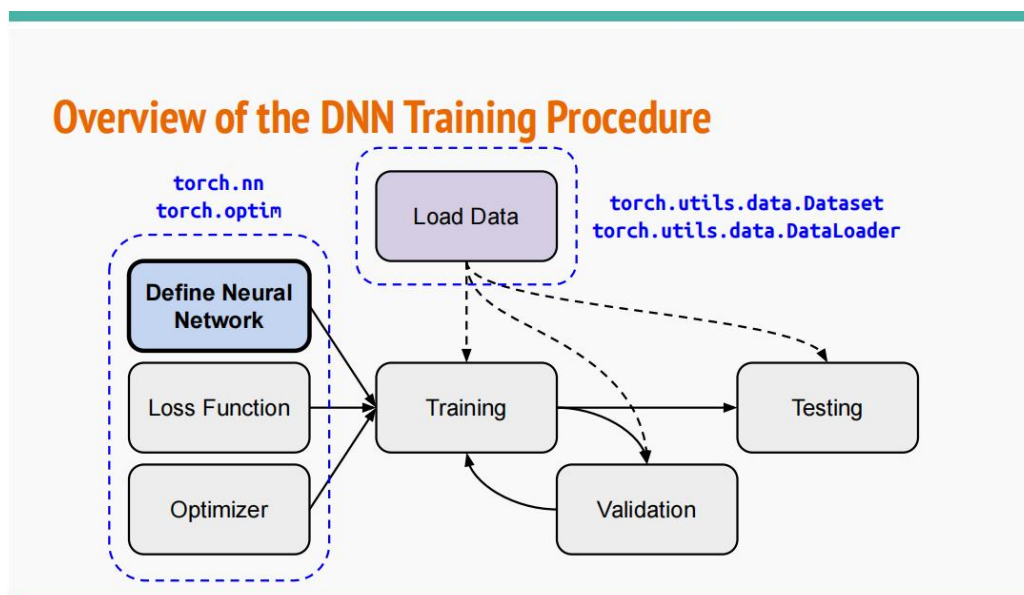
目录

1 总体流程	2
1.1 数据模块	2
1.2 网络模块	2
1.2.1 全连接神经网络	2
1.2.2 卷积神经网络	4
1.2.3 循环神经网络	8
1.2.4 生成模型	13
1.3 算法模块	14
1.3.1 牛顿法	14
1.3.2 动量法(Momentum)	15
1.3.3 牛顿动量法(Nesterov)	16
1.3.4 AdaGrad	16
1.3.5 RMSProp	17
1.3.6 Adam	17
1.4 训练测试模块	17
1.4.1 model.train()和 model.eval()	18
1.4.2 model.train()流程图	18
1.4.3 model.eval()流程图	18
1.5 可视化模块	18
2 注意力机制	19
2.1 查询键值	19
2.2 自注意力机制	21
2.3 交叉注意力机制	21

深度学习模型

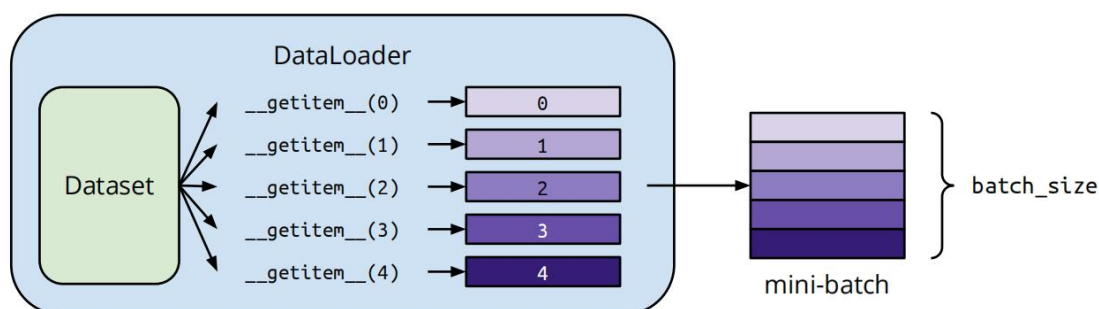
- 将大量不同类型神经网络层通过特定方式组合在一起背后的**数学原理**。
- 在原始数据上拟合极复杂的深层模型的**优化算法**。
- 有效训练模型，避免数值计算陷阱以及充分利用硬件性能所需的**工程技能**。
- 为解决方案挑选合适的变量（超参数）组合的**经验**。

1 总体流程



1.1 数据模块

- `dataset` 加载数据，`dataloader` 将加载好的数据以模型能够接收的方式输入到即将训练的模型中去。
- 根据输入输出，`dataset` 将数据集转换为 `src` 集和 `trg` 集(训练集和测试集都有)
`dataloader` 通过 `batch_size` 参数将训练集和测试集划分为 `Iteration` 个 `Batch`。



1.2 网络模块

1.2.1 全连接神经网络

记输入向量为 $x \in R^d$ ，神经网络的一个层把 x 映射为 $x' \in R^{d'}$ ，全连接层定义如下：

$$x' = \sigma(z), z = Wx + b$$

其中权重矩阵 $W \in R^{d' \times d}$ 和偏置向量 $b \in R^{d'}$ 是该层的参数，需要从数据中学习；

$\sigma(\cdot)$ 是激活函数，比如 softmax 函数，sigmoid 函数，ReLU 函数，最常用的是 ReLU，其定义如下：

$$\text{ReLU}(z) = [(z_1)_+, (z_2)_+, \dots, (z_{d'})_+]^T, (z_i)_+ = \max\{z_i, 0\}$$

我们称整个结构为全连接层，如图所示：

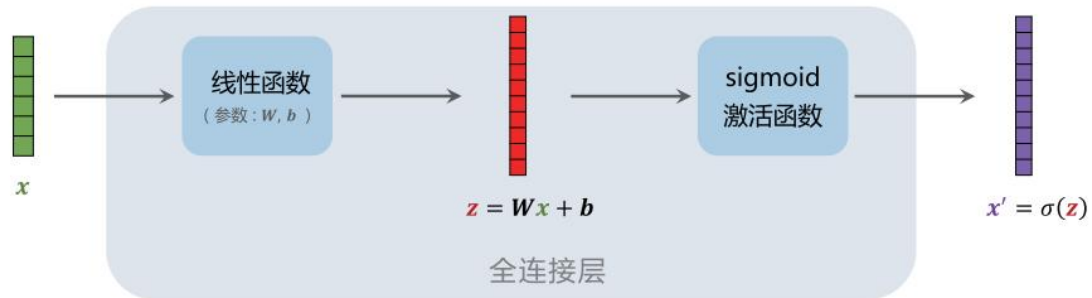
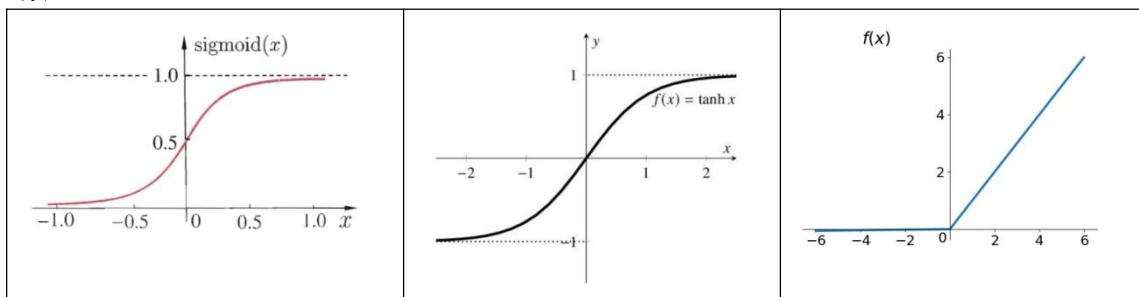


图 1-7 一个全连接层包括一个线性函数和一个激活函数

注意：全连接神经网络的超参数有全连接层的个数，每个全连接层的神经元个数，激活函数。



非线性激活函数：

- $\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$ ：将元素变换到 0-1 之间。
- $\text{tanh}(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$ ：将元素变换到 -1-1 之间。
- $\text{ReLU}(x) = \max(x, 0)$ ：保留正数元素，并将负数元素清零。

我们可以把全连接层当作基本组件，然后像搭积木一样搭建一个全连接神经网络，也叫多层感知机（multi-layer perceptron, MLP）：

$$\text{第 1 层: } x^{(1)} = \sigma_1(W^{(1)}x^{(0)} + b^{(1)})$$

$$\text{第 2 层: } x^{(2)} = \sigma_2(W^{(2)}x^{(1)} + b^{(2)})$$

...

$$\text{第 m 层: } x^{(m)} = \sigma_m(W^{(m)}x^{(m-1)} + b^{(m)})$$

其中 $W^{(1)}, \dots, W^{(m)}, b^{(1)}, \dots, b^{(m)}$ 是神经网络的参数，需要从训练数据中学习，不同层的参数是不同的，不同层的激活函数 $\sigma_1, \dots, \sigma_m$ 可以相同，也可以不同。

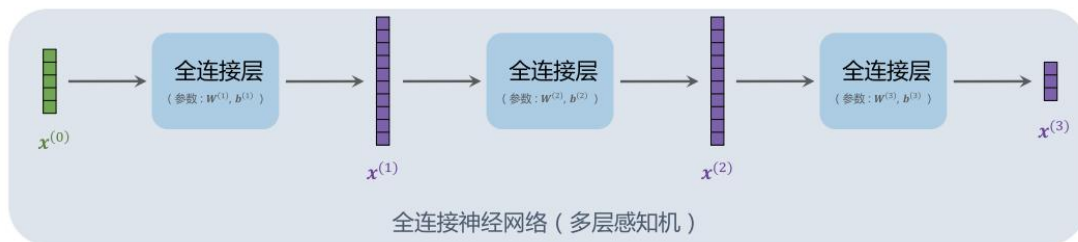


图 1-8 由 3 个全连接层组成的神经网络，每层有自己的参数

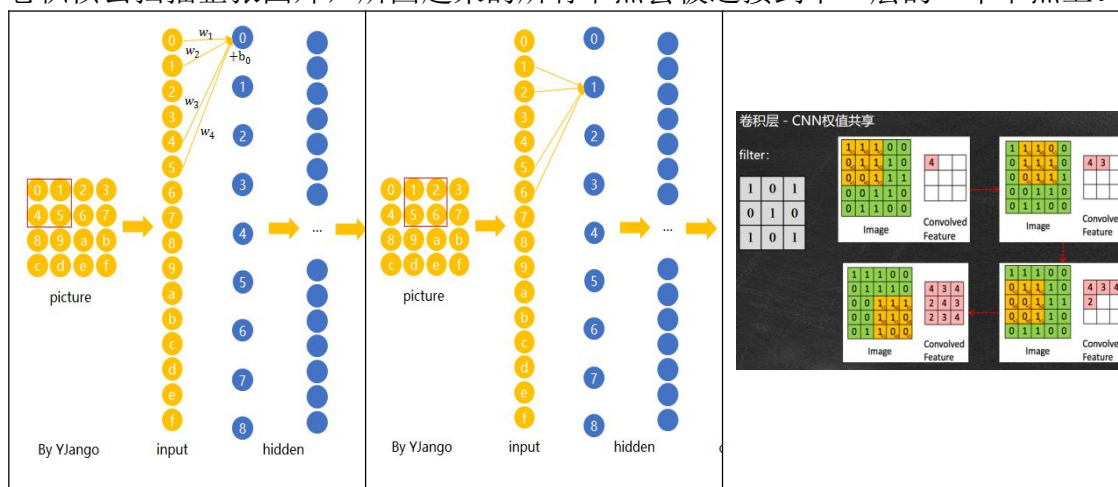
1.2.2 卷积神经网络

学者受生物学上的感受野机制的启发，提出了卷积神经网络（convolutional neural network, CNN），感受野机制主要是指听觉、视觉等神经系统中一些神经元的特性，即神经元只接收其所支配的刺激区域内的信号。基于该机制提出的 CNN 就是通过建立卷积层、池化层、全连接层实现对图像的精确处理。卷积层负责提取图像中的局部特征，接着利用池化层大幅降低参数数量从而提高训练效率，最后通过全连接层进行线性变换，输出结果。由于 CNN 在结构上具有局部连接、权值共享以及池化三个特性，使得网络具有一定程度的平移缩放和旋转不变性，可以保留图片的空间特性，因此在图像处理方面具有很大优势。

接下来简单介绍 CNN 各个层的基本原理：

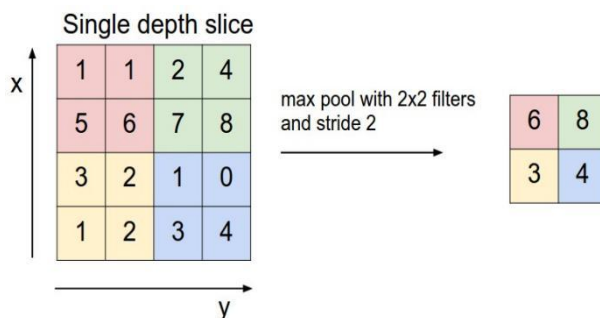
（1）卷积层：局部连接（卷积核跳跃），权值共享（卷积核权值）

先选择一个局部区域（filter）或者叫卷积核（大小 2×2 ，步长 1），用这个卷积核去扫描整张图片，所圈起来的所有节点会被连接到下一层的一个节点上。



（2）激活函数：将卷积层的结果经过激活函数映射来增强模型拟合能力。

（3）池化层：也叫下采样，用来压缩数据和参数的量，减小过拟合，通常取区域最大或平均值，如下最大池化核大小 2×2 ，步长 2。



• 特征不变性：通过池化操作，依然可以保留图像的重要特征，对图像进行池化只是去掉一些无关紧要的信息，留下的信息则具有尺度不变性的特征，可以充分

表达图像特征。

- 数据降维：即使经过卷积，图像的数据依然很大，包含很多重复的以及没有太大作用的信息，通过池化可以剔除这些冗余的信息，对数据起到降维的作用。

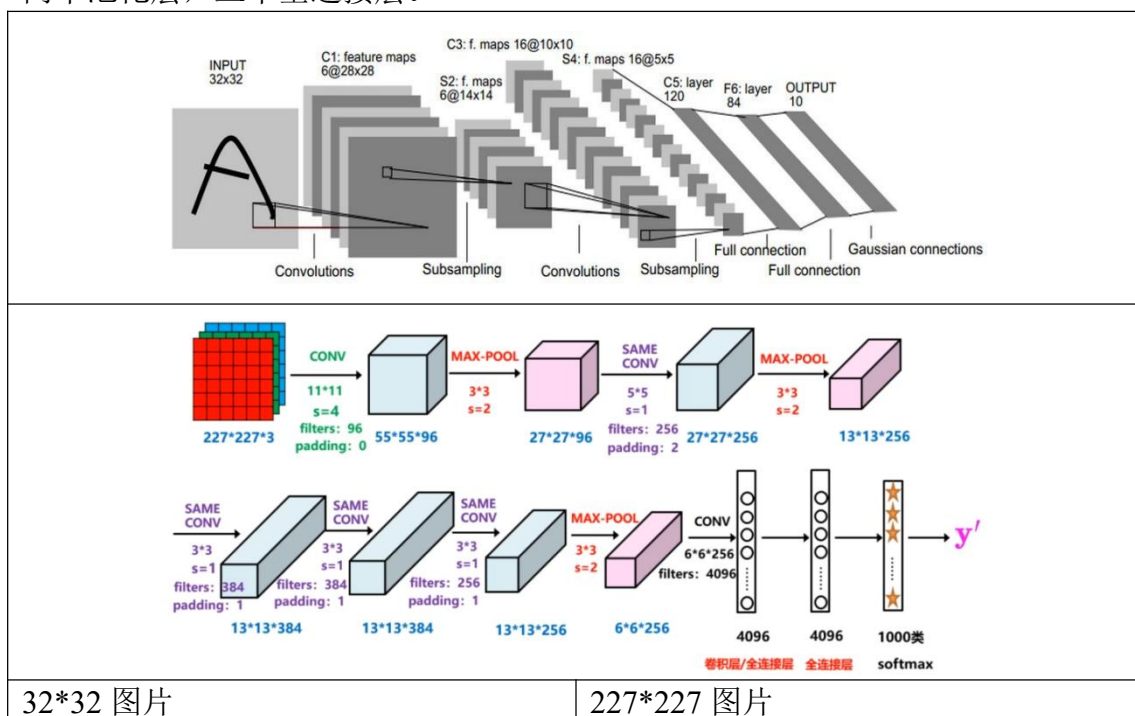
(4) 全连接层

经过卷积和池化处理后的数据输入到全连接层，根据回归和分类问题的需要，选择不同的激活函数获得最终想要的结果。

模型	特点
LeNet (98)	最早发布的卷积神经网络
AlexNet (12)	首次证明了学习到的特征可以超越手工设计的特征，一举打破了计算机视觉研究的现状。在 LeNet 的基础上，进行了一系列的改进，包括增加了 GPU 并行计算、dropout 层、Maxpooling 最大池化层、将激活函数改为 relu，层数也更加深
VGG 块	通过观察 LeNet、AlexNet 的结构，有大量重复的卷积层、全连接层、池化层等操作，于是诞生了将这些层合并成一个模块的思想，VGG 块也就产生了。
NiN 块	由一个卷积层和多个 1×1 的卷积层组成的块，该块允许更多的每像素非线性。去除了容易造成过拟合的全连接层，将它们替换为全局平均汇聚层（即在所有位置上进行求和）。NiN 的设计影响了许多后续卷积神经网络的设计
GoogLeNet (14)	吸收了 NiN 中串联网路的思想，并在此基础上做了改进。这篇论文的一个重点是解决了什么样大小的卷积核最合适的问题。
ResNet (15)	残差块，函数嵌套
DenseNet (17)	卷积块，函数分解

1.2.2.1 Lenet5/AlexNet

1998 年提出经典的卷积神经网络 Lenet5，具有一个输入层，两个卷积层，两个池化层，三个全连接层。



5*5 卷积核（6 个，步长 1，填充 0）	11*11 卷积核（96 个，步长 4，填充 0）
2*2 最大池化核（步长 2，填充 0）	3*3 最大池化核（步长 2，填充 0）
5*5 卷积核（16 个，步长 1，填充 0）	5*5 卷积核（256 个，步长 1，填充 2）
2*2 最大池化核（步长 2，填充 0）	3*3 最大池化核（步长 2，填充 0）
全连接层（120）	3*3 卷积核（384 个，步长 1，填充 1）
全连接层（84）	3*3 卷积核（384 个，步长 1，填充 1）
全连接层（10）	3*3 卷积核（256 个，步长 1，填充 1）
	3*3 最大池化核（步长 2，填充 0）
	全连接层（4096）
	全连接层（4096）
	全连接层（1000）

两个模型的区别：

- 在 AlexNet 的第一层，卷积窗口的形状是 11*11，由于 ImageNet 中大多数图像的宽和高比 MNIST 图像的多 10 倍以上，因此，需要一个更大的卷积窗口来捕获目标。第二层中的卷积窗口形状被缩减为 5*5，然后是 3*3。此外，在第一层、第二层和第五层卷积层之后，加入窗口形状为 3*3、步幅为 2 的最大汇聚层。而且，AlexNet 的卷积通道数目是 LeNet 的 10 倍。在最后一个卷积层后有两个全连接层，分别有 4096 个输出。这两个巨大的全连接层拥有将近 1GB 的模型参数。

- AlexNet 通过暂退法控制全连接层的模型复杂度，而 LeNet 只使用了权重衰减。为了进一步扩充数据，AlexNet 在训练时增加了大量的图像增强数据，如翻转、裁切和变色。这使得模型更健壮，更大的样本量有效地减少了过拟合。

AlexNet 的创新点：

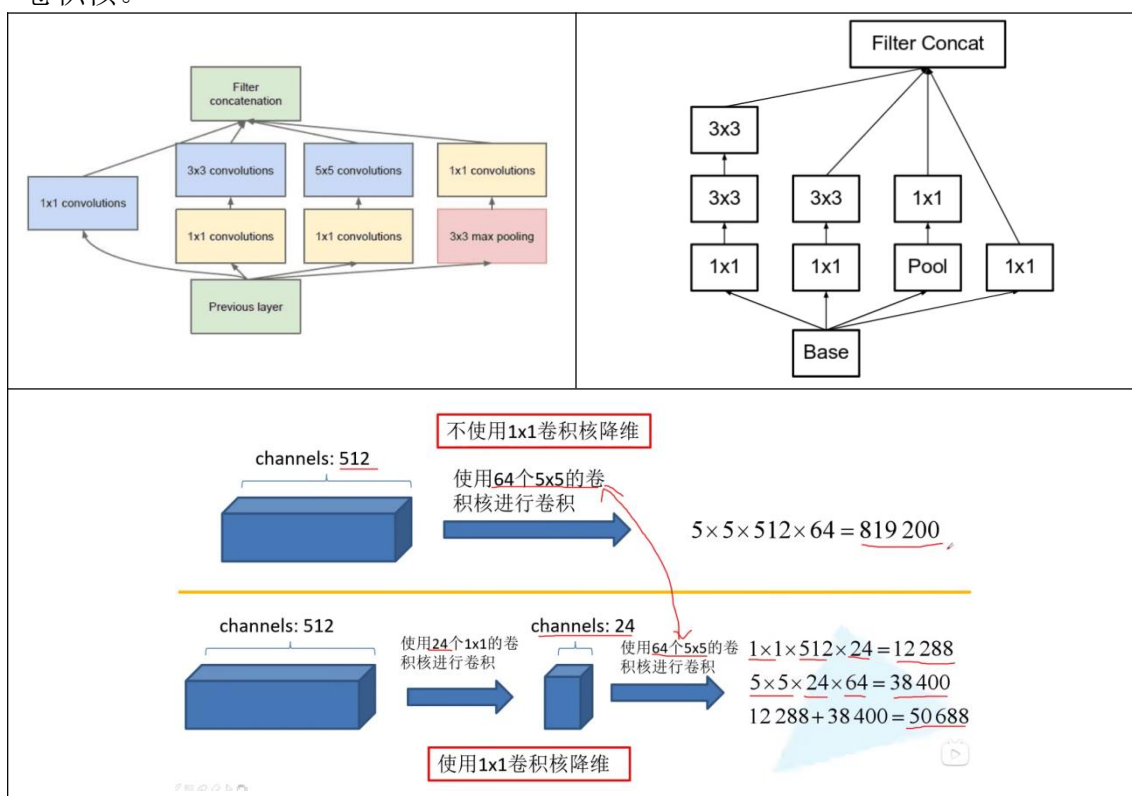
- 成功使用 ReLU 作为 CNN 的激活函数，并验证其效果在较深的网络超过了 Sigmoid，成功解决了 Sigmoid 在网络较深时的梯度弥散问题。虽然 ReLU 激活函数在很久之前就被提出了，但是直到 AlexNet 的出现才将其发扬光大。
- 训练时使用 Dropout 随机忽略一部分神经元，以避免模型过拟合。Dropout 虽有单独的论文论述，但是 AlexNet 将其实用化，通过实践证实了它的效果。在 AlexNet 中主要是最后几个全连接层使用了 Dropout。
- 在 CNN 中使用重叠的最大池化。此前 CNN 中普遍使用平均池化，AlexNet 全部使用最大池化，避免平均池化的模糊化效果。并且 AlexNet 中提出让步长比池化核的尺寸小，这样池化层的输出之间会有重叠和覆盖，提升了特征的丰富性。
- 数据增强，随机地从 256*256 的原始图像中截取 224*224 大小的区域（以及水平翻转的镜像），相当于增加了 $2 \times (256-224)^2 = 2048$ 倍的数据量。如果没有数据增强，仅靠原始的数据量，参数众多的 CNN 会陷入过拟合中，使用了数据增强后可以大大减轻过拟合，提升泛化能力。进行预测时，则是取图片的四个角加中间共 5 个位置，并进行左右翻转，一共获得 10 张图片，对他们进行预测并对 10 次结果求均值。同时，AlexNet 论文中提到了会对图像的 RGB 数据进行 PCA 处理，并对主成分做一个标准差为 0.1 的高斯扰动，增加一些噪声，这个 Trick 可以让错误率再下降 1%。

1.2.2.2 GoogleNet

本文的一个观点是，有时使用不同大小的卷积核组合是有利的。本节将介绍一个稍微简化的 GoogLeNet 版本：我们省略了一些为稳定训练而添加的特殊特性，现在有了更好的训练方法，这些特性不是必要的。

网络的创新点：

- 引入 Inception 结构，融合不同尺度的特征信息（height, width 相同，在 channel 深度方面进行拼接），并使用 1x1 的卷积核进行降维以及映射处理。在 GoogLeNet 中，基本的卷积块被称为 Inception 块，Inception 模块原先是 1*1, 3*3, 5*5 的卷积核，3*3 的最大池化核，来提取不同尺度的特征，为了减少通道数和参数，增加了 3 个 1*1 卷积核，为了减少参数，将 5*5 卷积核替换为 2 个连续的 3*3 卷积核。



- 丢弃全连接层，使用平均池化层，大大减少模型参数。
- 批量规范化使得研究人员能够训练 100 层以上的网络。

批量规范化应用于单个可选层（也可以应用到所有层），在每次训练迭代中，将输入减去其均值并除以其标准差，然后应用比例系数和比例偏移（模型参数）。在应用批量规范化时，**批量大小的选择**可能比没有批量规范化时更重要。

假设 $x \in B$ 表示来自批量 B 的一个输入 x ，批量规范化 BN 表示：

$$BN(x) = \gamma \frac{x - \mu_B}{\sigma_B} + \beta$$

其中， $\hat{\mu}_B = \frac{1}{|B|} \sum_{x \in B} x$ ， $\hat{\sigma}_B^2 = \frac{1}{|B|} \sum_{x \in B} (x - \hat{\mu}_B)^2 + \varepsilon$ 。

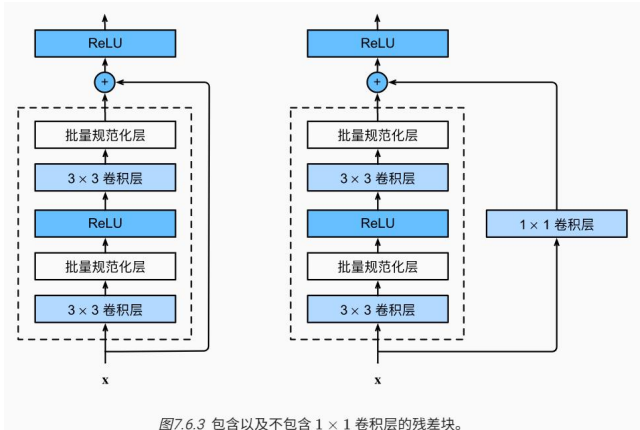
对于全连接层，批量规范化层置于全连接层中的仿射变换和激活函数之间：

$$h = \phi(BN(Wx + b))$$

对于卷积层，批量规范化置于卷积层和激活函数之间，并且卷积层会有多个通道，不同的批量在同一通道具有相同的比例系数和比例偏移。

1.2.2.3 ResNet

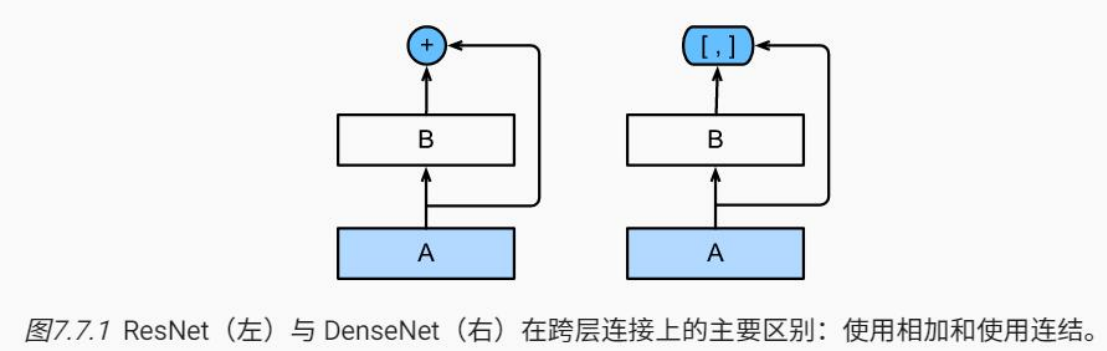
当神经网络层数过多时，在反向传播的时候容易造成梯度消失，残差网络可以改善这个问题。残差块有两种定义方式：



每个残差模块有 4 个卷积层（不包括恒等映射的卷积层），加上第一个卷积层和最后一个全连接层，共有 18 层。因此，这种模型通常被称为 ResNet-18。通过配置不同的通道数和模块里的残差块数可以得到不同的 ResNet 模型，例如更深的含 152 层的 ResNet-152。虽然 ResNet 的主体架构跟 GoogLeNet 类似，但 ResNet 架构更简单，修改也更方便。这些因素都导致了 ResNet 迅速被广泛使用。

1.2.2.4 DenseNet

稠密连接网络在某种程度上是 ResNet 的逻辑扩展，ResNet 将函数展开为 $f(x)=x+g(x)$ 线性项加非线性项，如果我们想将 f 拓展成超过两部分的信息呢？一种方案便是 DenseNet。



在应用越来越复杂的函数序列后，我们执行从 x 到其展开式的映射：

$$x \rightarrow [x, f_1(x), f_2(x, f_1(x)), f_3(x, f_1(x), f_2(x, f_1(x))), \dots]$$

最后，将这些展开式结合到多层感知机中，再次减少特征的数量。实现起来非常简单：我们不需要添加术语，而是将它们连接起来。DenseNet 这个名字由变量之间的“稠密连接”而得来，最后一层与之前的所有层紧密相连。

1.2.2.5 模型参数和超参数

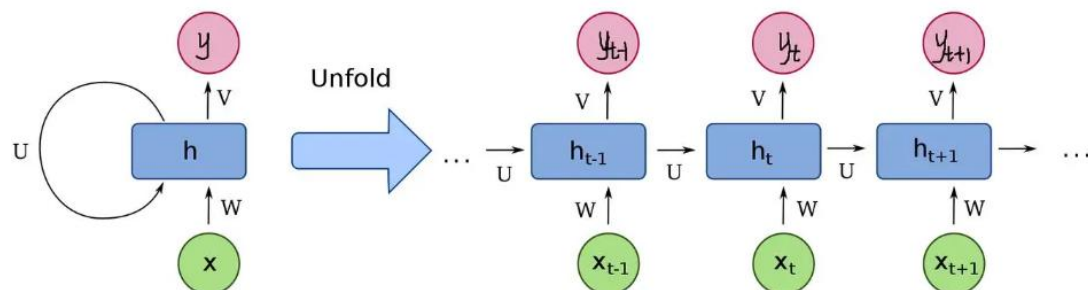
- 模型参数：卷积核，池化核，全连接层的权重和偏差。
- 超参数：通道数（卷积核、池化核的数量），卷积核，池化核的大小，全连接层的个数、神经元个数，激活函数类型，函数中的超参数。

1.2.3 循环神经网络

1.2.3.1 RNN

人类在语言表达时，每个字都与前面表达的内容有逻辑顺序，需要在记住前面关键信息的基础上，理解后面的表达，例如无论是 arrive Beijing 还是 leave Beijing, Beijing 作为一般神经网络的输入时，输出的都是 Destination，我们希望神经网络有记忆，记得 Beijing 前的 arrive 或者 leave，该怎么实现呢？

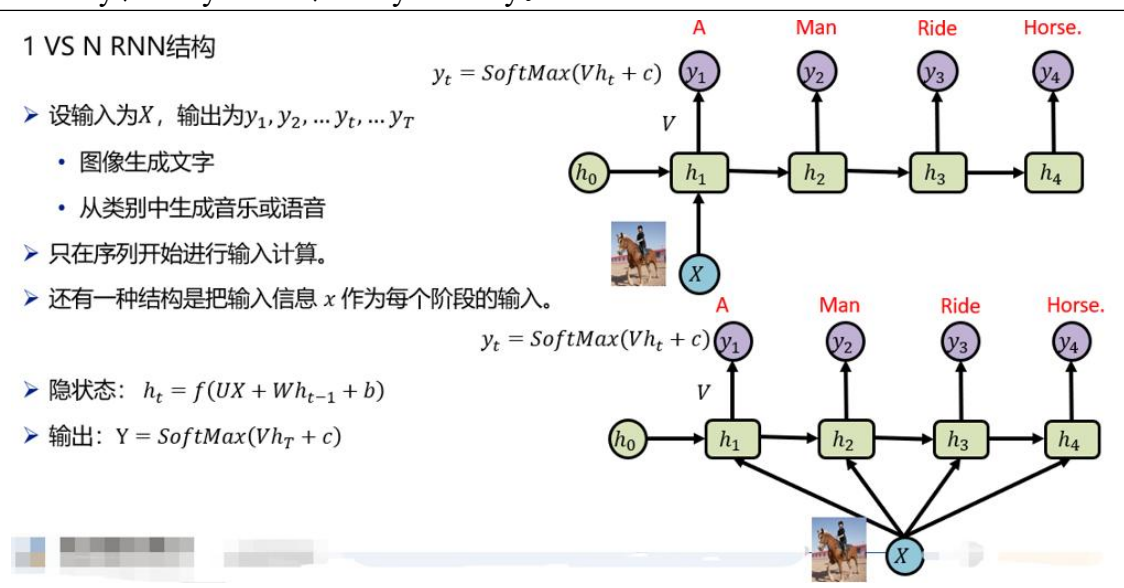
为更好地处理时间序列问题，学者提出了循环神经网络结构（Recurrent Neural Network, RNN），最基本的由输入层、一个隐藏层、一个输出层组成

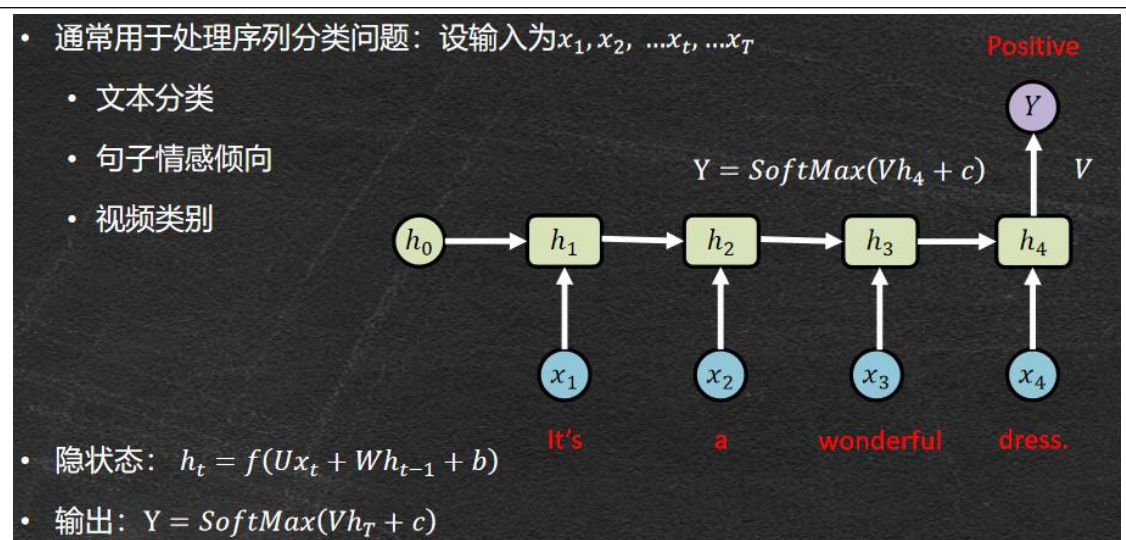


其中 W 是输入层到隐藏层的权值矩阵， V 是隐藏层到输出层的权值矩阵， U 是隐藏层上一时刻的值作为当前时刻输入的权值矩阵，把左边结构按时间线展开，得到右边结构。

网络在 t 时刻接收到输入 x_t 后，隐藏层的值是 h_t ，输出值是 y_t ， h_t 的值不仅取决于 x_t ，还取决于 h_{t-1} ，因此，在 RNN 结构下，当前时刻的信息在下一时刻也会被输入到网络中，网络中的信息形成了时间相关性，解决了处理时间序列的问题。神经网络模型“学”到的东西隐含在权值 U 中。基础的神经网络只在层与层之间建立全连接，而 RNN 与它们最大的不同之处在于同一层内的神经元在不同时刻也建立了全连接，即 U 与时间有关。

根据不同的输入输出模式，可以将 RNN 分为以下四种结构：one to one、one to many、many to one、many to many。





1.2.3.2 LSTM

在 RNN 中，因为通常前期的层会因为梯度消失而停止学习，RNN 会忘记它在更长的序列中看到的東西，从而只拥有短期记忆。(sigmoid 最大梯度 0.25)，因此提出 LSTM (Long Short-Term Memory, 97) 和 GRU (Gated Recurrent Unit, 14)。

new weight = weight - learning rate*gradient

$$\boxed{2.0999} = \boxed{2.1} - \boxed{0.001}$$

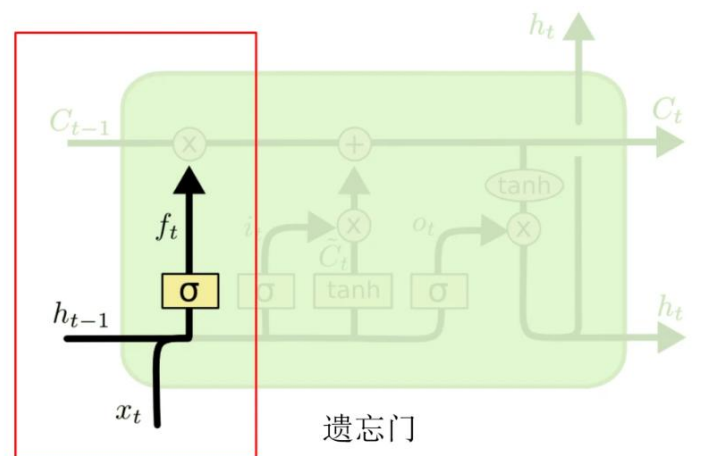
Not much of a difference update value

RNN 和 LSTM 的最大区别在于分布在隐藏层的神经元结构，传统 RNN 的神经元结构简单，例如只包含一个激活函数层。LSTM 的神经元结构更加复杂，增加了状态 c ，称为单元状态 (Cell State)，用来保存长期的状态，并用三个门去除和增加单元状态中信息。

下面对这三个门逐一介绍，在以下示意图中，黄色矩形是学习得到的神经网络层，粉色圆形表示运算操作，箭头表示向量的传输过程。

(1) 遗忘门 f_t : 决定上一时刻的单元状态 c_{t-1} 有多少保留到当前时刻状态 c_t 。式中， W_f 是遗忘门的权值矩阵， $[h_{t-1}, x_t]$ 表示把两个向量连接成一个更长的向量， b_f 是遗忘门的偏置项， σ 是 Sigmoid 函数。

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

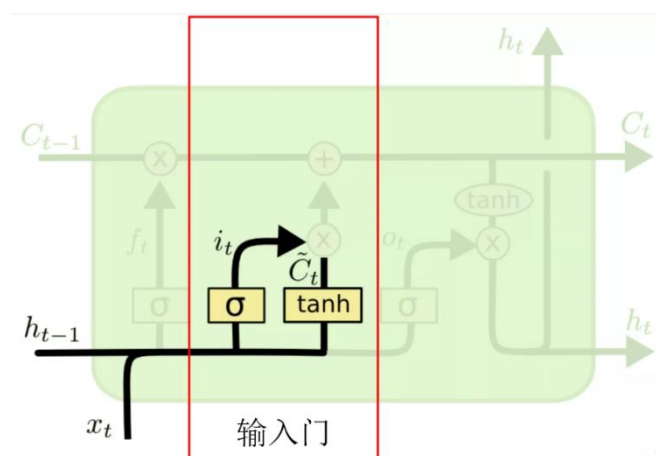


在遗忘门中，来自先前隐状态的信息和来自当前输入的信息加权后传递到 sigmoid 函数，并将值压缩到 0 和 1 之间。越接近 0 意味着丢弃，越接近 1 意味着保留。

(2) 输入门：决定当前时刻网络的输入 x_t 有多少保存到单元状态 c_t 。

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

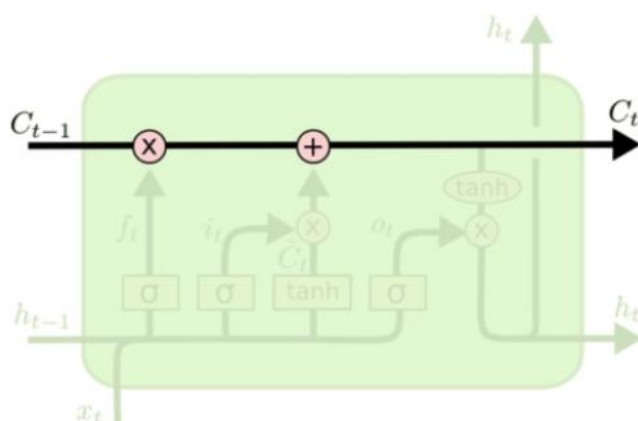
$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$



在输入门中，来自先前隐状态的信息和来自当前输入的信息加权后传递到 sigmoid 函数，并将值压缩到 0 和 1 之间。越接近 0 意味着丢弃，越接近 1 意味着保留。还可以传递给 tanh 函数，使值变为 -1 到 1 之间的值，以帮助调节神经网络。

(3) 更新单元状态：在遗忘门的控制下，网络可以保存很久之前的信息，在输入门的控制下，无用信息无法进入到网络当中。

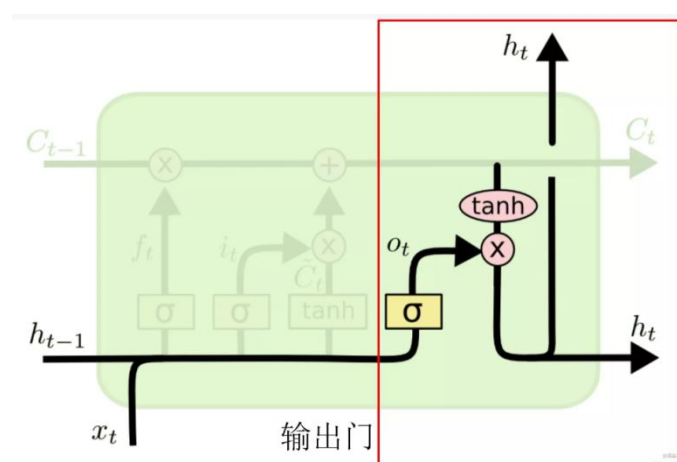
$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$



(4) 输出门：决定 c_t 有多少输入到当前输出值 h_t 中。

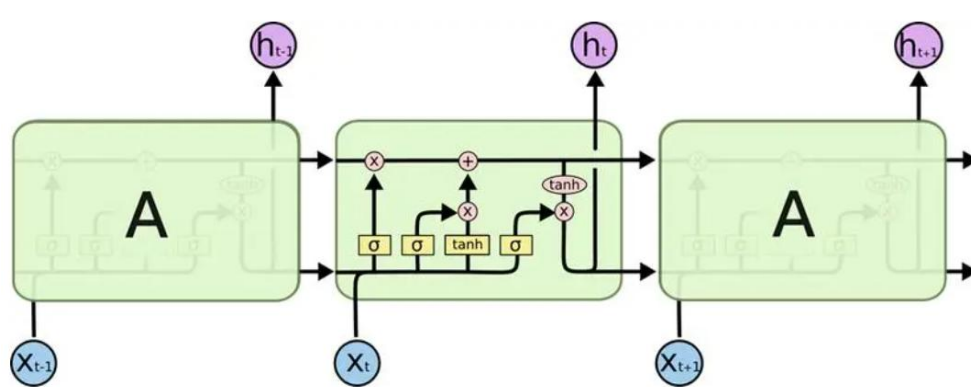
$$O_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = O_t \tanh(C_t)$$



在输出门中，来自先前隐状态的信息和来自当前输入的信息加权后传递到 sigmoid 函数，并将值压缩到 0 和 1 之间。越接近 0 意味着丢弃，越接近 1 意味着保留。然后将新修改的单元状态传递给 tanh 函数，将 tanh 输出与 sigmoid 输出相乘，以确定隐状态应该包含的信息。

(5) 重复：新的单元状态和新的隐藏状态随后被转移到下一步中。



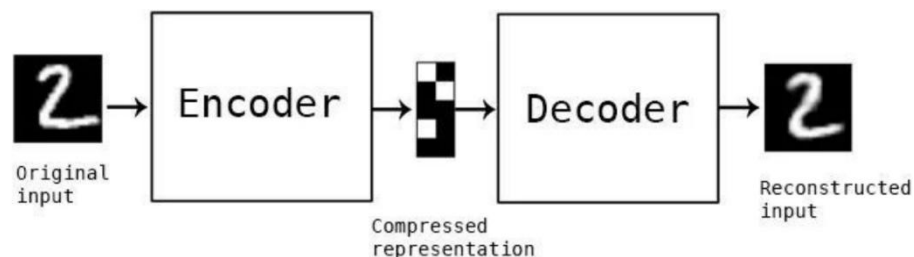
注：LSTM 与 GRU 的主要缺点是计算过程中会产生梯度爆炸，为解决这个问题，一是对梯

度进行裁剪，限制梯度的模长在一定范围内，另一方面对激活函数进行归一化处理，提高模型在训练过程中的稳定性。

1.2.4 生成模型

生成模型：给定一组随机数，根据随机数来生成服从训练数据分布的数据。主要有 VAE 和 GAN：VAE 使用两个神经网络，Encoder 将输入变为隐变量，这个隐变量的值要加上一定的限制，使之服从目标分布；Decoder 将隐变量变为目标值。训练完成的模型只需要使用 Decoder，通过输入随机产生的隐变量的值，输出目标值；GAN 使用两个神经网络，Generator 输入服从一定分布的隐变量，产生隐变量对应的（图像）；Discriminator 是判断输入的（图像）来源于训练集还是 Generator。

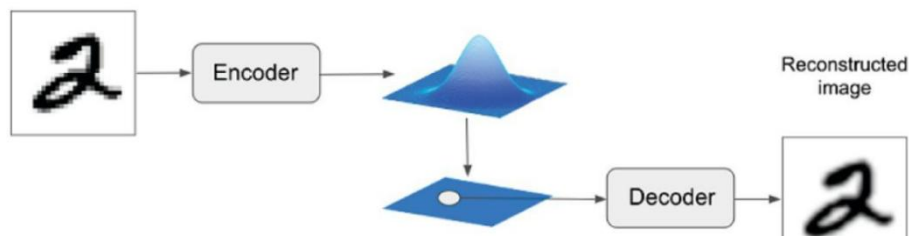
(1) AE



对输入图像进行逐步降采样，输出降采样的中间编码表示（隐含变量），然后对中间编码表示进行升采样，输出最后的预测图像。在实际实现中，编码器可以使用卷积模块结合池化模块来实现，解码器可以使用卷积模块结合转置卷积来实现。

(2) VAE

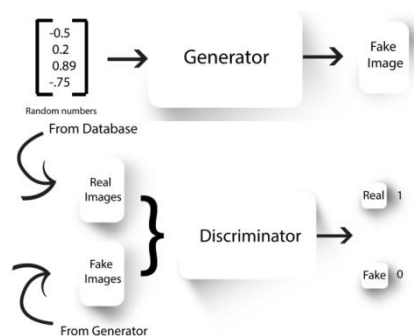
意思是 AE 产生服从给定分布的隐变量，VAE 产生隐变量服从的参数，然后使用这些参数产生服从给定分布的隐变量。



中间细节虽然不清楚，但是是用 KL 散度来衡量两个分布（Q 为目标分布，P 为隐含变量的分布）之间的差异性：

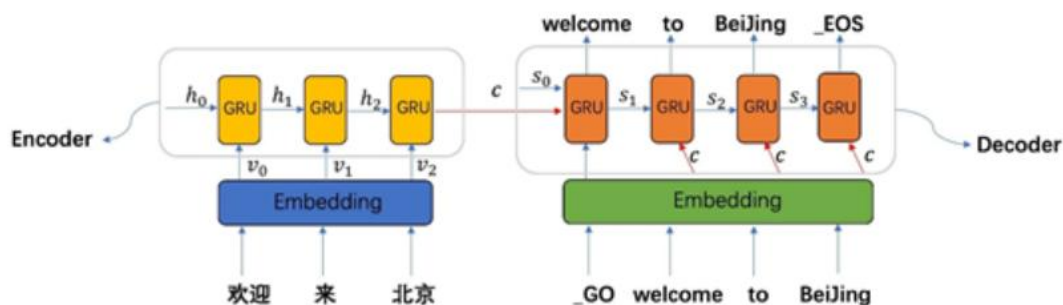
$$D_{KL}(P \parallel Q) = \int_{-\infty}^{+\infty} p(x) \log \frac{p(x)}{q(x)} dx$$

(3) GAN



(4) Seq2Seq

将一个文本序列转换为另一个文本序列的任务为 Seq2Seq。



首先，解码器输入一个特殊的单词，即句子开头单词（标记为<SOS>），输出第一个预测的单词，然后根据第一个预测的单词进行第二个单词的预测，直到达到最大预测长度或者预测得到另一个特殊单词（标记为<EOS>）。

由于 RNN 的特性，对于很长的序列，包括 LSTM 和 GRU（设计上只是尽可能减少遗忘发生）都不可避免出现遗忘情况，这个缺点可以使用注意力机制来解决。注意力机制就是通过引入一个神经网络，计算编码器输出对解码器贡献权重，最后计算加权平均后编码器的输出。

1.3 算法模块

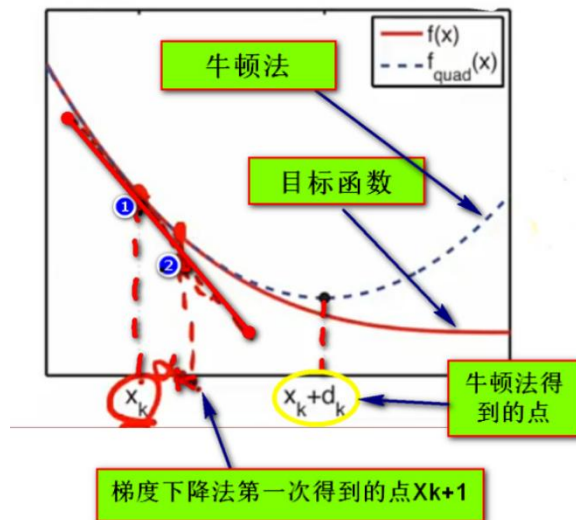
1.3.1 牛顿法

$$W = W - \frac{J'(W)}{J''(W)}$$

对抛物线 $f(x)$ 来说，如何快速下降？

$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2} f''(x_0)(x - x_0)^2$ 求极值，则

$$f'(x) = f'(x_0) + f''(x_0)(x - x_0) = 0, \quad x = x_0 - \frac{f'(x_0)}{f''(x_0)}.$$



对于多变量的情况， $W = W - \nabla^2 J(W)^{-1} \nabla J(W)$

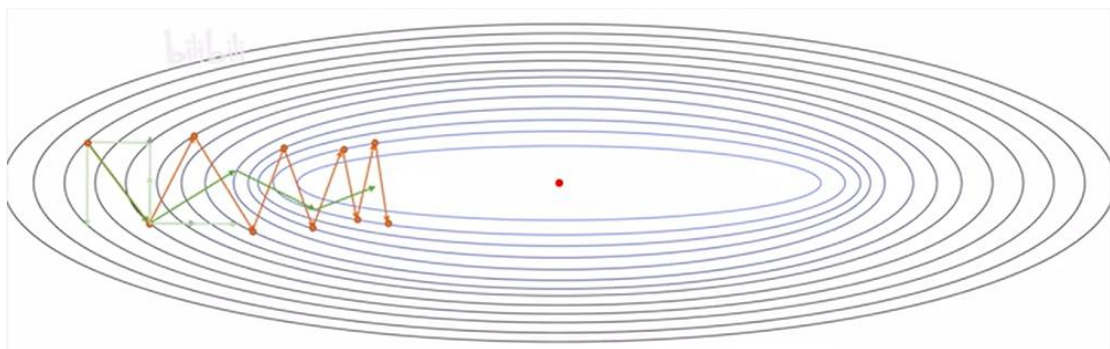
其中 $\nabla^2 J(W)^{-1}$ 是 Hessian 矩阵：

$$\begin{bmatrix} \frac{\partial}{\partial W_1} \frac{\partial J}{\partial W_1} & \frac{\partial}{\partial W_2} \frac{\partial J}{\partial W_1} & \cdots & \frac{\partial}{\partial W_n} \frac{\partial J}{\partial W_1} \\ \frac{\partial}{\partial W_1} \frac{\partial J}{\partial W_2} & \frac{\partial}{\partial W_2} \frac{\partial J}{\partial W_2} & \cdots & \frac{\partial}{\partial W_n} \frac{\partial J}{\partial W_2} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial}{\partial W_1} \frac{\partial J}{\partial W_n} & \frac{\partial}{\partial W_2} \frac{\partial J}{\partial W_n} & \cdots & \frac{\partial}{\partial W_n} \frac{\partial J}{\partial W_n} \end{bmatrix}$$

Hessian 矩阵非正定（非凸）导致无法收敛；Hessian 矩阵维度过大带来巨大的计算量；因此牛顿法只是理论上可行的方法，实际上很难操作。

1.3.2 动量法(Momentum)

在 SGD 基础上改进，以前的改变是有惯性的，要想发生改变，先将他们抵消：梯度在纵轴上的分量会来回震荡，将这种情况减弱，在横轴上的分量一直指向前，想办法加速，方法就是利用历史数据来修正，如果 x_1 处的纵轴分量向下， x_2 处的纵轴分量向上，方向相反，就会减少这个维度的量；如果 x_1 处的横轴分量向前， x_2 处的横轴分量向前，方向相同，就会增加这个维度的量。



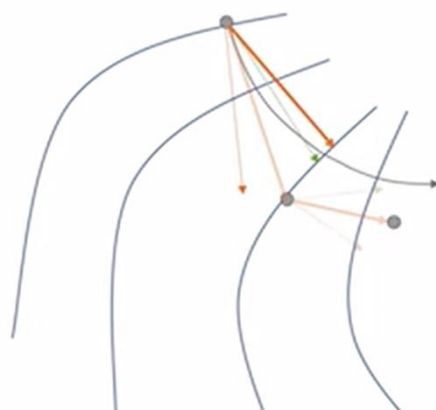
$$\begin{bmatrix} b_{(t)} \\ W_{(t)1} \\ W_{(t)2} \\ \dots \\ W_{(t)i} \end{bmatrix} = \begin{bmatrix} b_{(t-1)} - \eta \frac{\partial J}{\partial b} \\ W_{(t-1)1} - \eta \frac{\partial J}{\partial W_1} \\ W_{(t-1)2} - \eta \frac{\partial J}{\partial W_2} \\ \dots \\ W_{(t-1)i} - \eta \frac{\partial J}{\partial W_i} \end{bmatrix}$$

令 $\nabla W_{(t-1)i} = \frac{\partial J(W_{(t-1)i})}{\partial W_i}$, 令 $V_{(t-1)} = \beta V_{(t-2)} + (1 - \beta) \nabla W_{(t-1)i}$ (距离越近的时刻权重越大, 既能用历史数据进行修正, 又让久远的历史数据对当前影响非常小)

$$W_{(t)i} = W_{(t-1)i} - \eta V_{(t-1)} \circ$$

1.3.3 牛顿动量法(Nesterov)

超前修正, 从左看, 橙黄是梯度方向, 浅绿是冲量方向, 则中间粗的橙黄是修正的梯度方向。



然后再是梯度方向, 冲量方向, 修正的梯度方向, 如果有预知未来的能力, 知道第二步的位置, 那么第一步修正的程度就大了, 就会有更好的效果。

$$\nabla W_{(t-1)i} = \frac{\partial J(W_{(t-1)i} + \gamma V_{(t-2)})}{\partial W_i}$$

上面的方法对所有的参数都是同一个学习率, 现在对不同的参数可以通过逐参数适应学习率方法设置不同的学习率。

1.3.4 AdaGrad

AdaGrad 将学习率和参数相适应, 对频繁参数的罕见更新和较小更新执行更大的更新, 增加了罕见但信息丰富的特征的影响, 因此非常适合处理稀疏数据。但其中一个缺点是: 在深度学习中单调的学习率被证明通常过于激进且过早停止

学习。

该算法的基本思想是对每个变量采用不同的学习率，这个学习率在一开始比较大，用于快速梯度下降；随着优化过程进行，对于已经下降很多的变量，减缓学习率；对于还没怎么下降的变量，则保持较大的学习率，其更新参数的方式如下：

$$W_{(t)i} = W_{(t-1)i} - \frac{\eta}{\sqrt{S_{(t-1)}} + \varepsilon} \nabla W_{(t-1)i}$$

其中， $\nabla W_{(t-1)i} = \frac{\partial J(W_{(t-1)i})}{\partial W_i}$ ， $S_{(t-1)} = S_{(t-2)} + \nabla W_{(t-1)i} \nabla W_{(t-1)i}$ （所有历史数据梯度的平方，梯度大，下的快，减的快）， ε 避免分母为 0。

1.3.5 RMSProp

RMSProp 是 AdaGrad 的一种改进，鉴于神经网络都是非凸条件下的，RMSProp 在非凸条件下结果更好，改变梯度累积为指数衰减的移动平均。该算法不像 AdaGrad 直接累加平方梯度，而是加了一个衰减系数来控制历史信息的获取多少，即做了一个梯度平方的滑动平均，具体区别如下：

AdaGrad 的历史梯度：

$$S_{(t-1)} = S_{(t-2)} + \nabla W_{(t-1)i} \nabla W_{(t-1)i}$$

RMSProp 的历史梯度：

$$S_{(t-1)} = \beta S_{(t-2)} + (1 - \beta) \Delta W_{(t-1)i} \Delta W_{(t-1)i}$$

1.3.6 Adam

Adam 算法即自适应时刻估计方法（Adaptive Moment Estimation），相当于动量法和 RMSProp 相结合，能够计算每个参数的自适应学习率，将惯性保持和环境感知这两个优点集于一身。

$$V_{(t-1)} = \beta_1 V_{(t-2)} + (1 - \beta_1) \Delta W_{(t-1)i}$$

$$\Delta W_{(t-1)i} = \frac{\partial J(W_{(t-1)i})}{\partial W_i}$$

$$S_{(t-1)} = \beta_2 S_{(t-2)} + (1 - \beta_2) \Delta W_{(t-1)i} \Delta W_{(t-1)i}$$

$$W_{(t)i} = W_{(t-1)i} - \frac{\eta}{\sqrt{S_{(t-1)}} + \varepsilon} V_{(t-1)}$$

1.4 训练测试模块

在模型初始化完成后，我们交替地进行正向传播和反向传播，首先根据初始化模型参数正向计算样本预测值，进而计算损失函数；然后反向计算损失函数的梯度，进而更新模型参数；如此循环直至 loss 收敛或达到最大 epochs。

如何对超参数 epoch 进行设置，取值过小可能会导致欠拟合，取值过大可能会导致过拟合，而 EarlyStopping 可以解决 Epoch 需要手动设定的问题，也可以认为是一种避免网络发生过拟合的正则化方法。

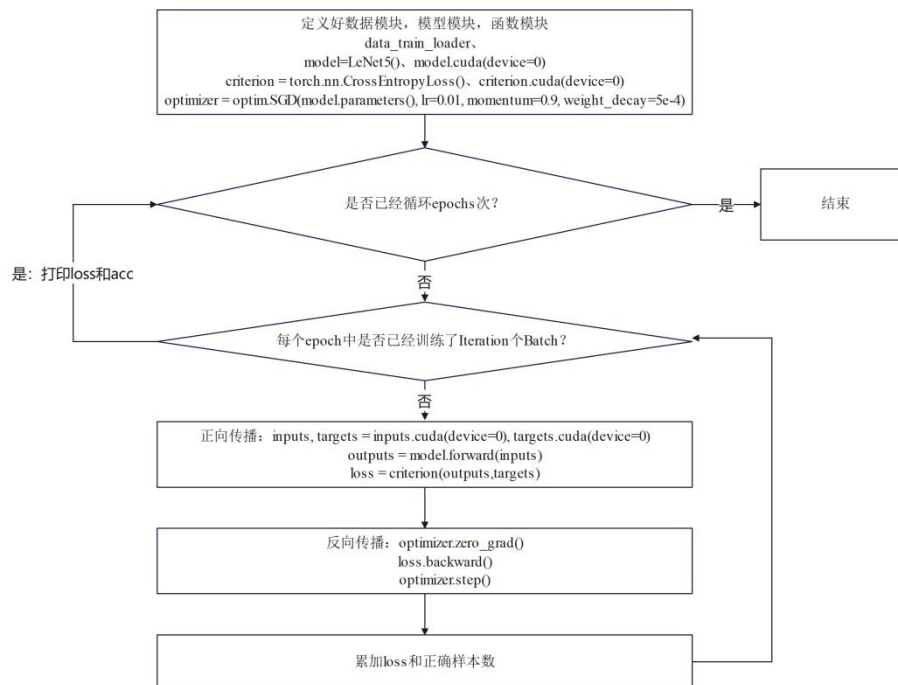
其原理大致分为三个部分：将原数据分为训练集和验证集；只在训练集上进行训练，并每隔一个周期计算模型在验证集上的误差，如果随着周期的增加，在验证集上的误差也在增加，则停止训练；将停止之后的权重作为网络的最终参数。

1.4.1 model.train()和 model.eval()

model.train()用于训练阶段，model.eval()用于验证和测试阶段，两者区别是对于 Dropout 和 Batch-Normalization 层(BN 层)的影响。

在 model.train()模式下，Dropout 层（常常用于抑制过拟合，参数 p 指该层神经元在每次迭代训练时会随机有 $1-p$ 的可能性被丢失）会按照设定参数 p 设置保留激活神经元的概率；BN 层会继续计算数据的均值和方差等参数并更新；在 model.eval()模式下，Dropout 层设定的参数无效，所有的激活神经元都可以通过该层，同时 BN 层会停止计算均值和方差，直接使用在训练阶段已经学习好的均值和方差运行模型。

1.4.2 model.train()流程图



注：超参数 epochs 设置过小会导致欠拟合，设置过大会导致过拟合，可以参考 EarlyStopping。

1.4.3 model.eval()流程图

比着上面没有了反向传播过程。

1.5 可视化模块

在模型训练过程中，有时不仅需要保存已经训练好的模型，也需要将训练过程中的训练集损失函数，验证集损失函数，模型计算图（即模型框架图、模型数据流图）等保存下来，供后续分析作图使用。例如，通过损失函数变化情况，可以观察模型是否收敛，通过模型计算图，可以观察数据流动情况等。

TensorBoard 是一个数据可视化工具，能直观地显示损失函数的下降趋势是否合理，张量分量的分布是否在训练过程中发生变化等。可以将数据，模型计算图等可视化，会自动获取最新的数据信息，将其存入日志文件中，并且会在日志文件中更新信息，运行数据或模型最新的状态。

```
pip install tensorflow-tensorboard==1.5.1
```

```
pip install tensorboard==1.15.0
```

然后在训练模型的代码中加入 TensorBoard 写入数据的相关代码，运行后会发现在当前目录下多了一个文件夹 runs，命令行运行 `tensorboard --logdir runs` 命令，但是复制网址 <http://autodl-container-c1bc119952-c3fddc16:6006/> 进浏览器打不开，本地电脑可以。将日志文件保存进 tf-logs 就行。

通过 TensorBoard 可以分析模型训练的行为是否正确：

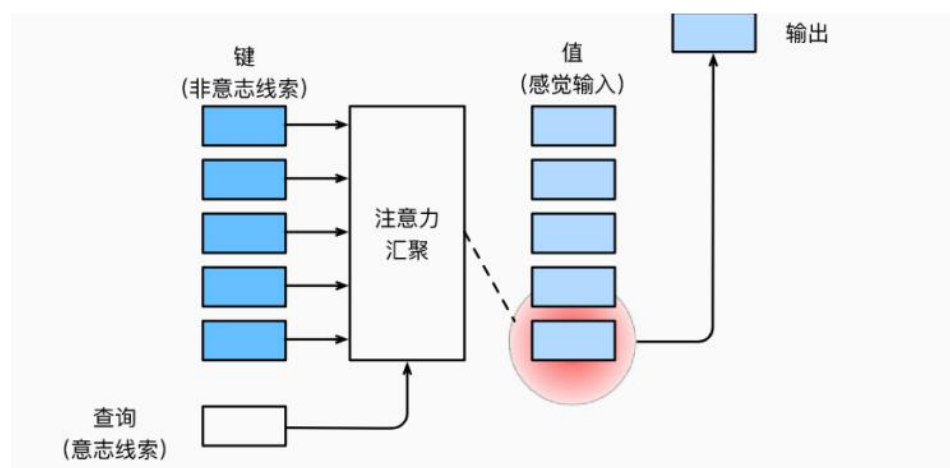
如果损失函数上升或者振荡，说明学习率过大，如果损失函数下降缓慢，说明学习率偏小；随着训练的进行，参数的分布和直方图应该不断发生变化，直到分布稳定为止，如果在训练过程中参数的分布没有变化，可能模型结构有问题，或者梯度无法在模型中得到反向传播。

2 注意力机制

2.1 查询键值

注意力有两种：非自主性的（提示是基于环境中物体的突出性和易见性，比如颜色）和自主性的（喝咖啡后，我们会变得兴奋并想读书，所以转过头，重新聚焦眼睛，然后看看书）。

注意力机制三要素：查询 Query、键 Key，值 Value，注意力机制通过注意力汇聚将查询（自主性提示）和键（非自主性提示）结合在一起，实现对值（感官输入）的选择倾向。



Nadaraya-Watson 核回归：具有注意力机制的机器学习范例。

$$f(x) = \frac{\sum_{i=1}^n \frac{K(x-x_i)}{\sum_{j=1}^n K(x-x_j)} y_i}{\sum_{j=1}^n K(x-x_j)}$$

其中 x 是查询， (x_i, y_i) 是键值对，注意力汇聚是 y_i 的加权平均。将查询 x 和键 x_i

之间的关系建模为注意力权重 $\alpha(x, x_i)$ ，这个权重将被分配给每一个对应值 y_i 。

对于任何查询，模型在所有键值对注意力权重都是一个有效的概率分布：它们是非负的，并且总和为 1。下面以一个高斯核为例：

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{u^2}{2}}$$

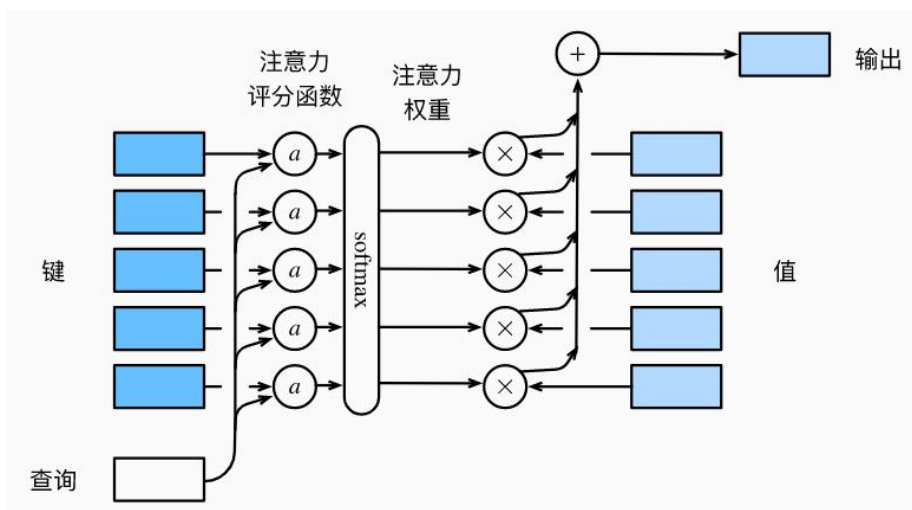
$$f(x) = \sum_{i=1}^n \frac{e^{-\frac{(x-x_i)^2}{2}}}{\sum_{j=1}^n e^{-\frac{(x-x_j)^2}{2}}} y_i = \sum_{i=1}^n \text{soft max}(-\frac{(x-x_i)^2}{2}) y_i$$

如果一个键 x_i 越是接近给定的查询 x ，那么分配给这个键对应值 y_i 的注意力权重就会越大，也就“获得了更多的注意力”。（测试数据的输入相当于查询，而训练数据的输入相当于键。因为两个输入都是经过排序的，因此由观察可知“查询-键”对越接近，注意力汇聚的注意力权重就越高。）（是不是意味着查询-键的相似性，注意力权值可以通过注意力评分函数来计算）

非参数的 Nadaraya-Watson 核回归具有一致性（consistency）的优点：如果有足够的数据，此模型会收敛到最优结果。尽管如此，我们还是可以轻松地将可学习的参数集成到注意力汇聚中。

$$f(x) = \sum_{i=1}^n \frac{e^{-\frac{((x-x_i)w)^2}{2}}}{\sum_{j=1}^n e^{-\frac{((x-x_j)w)^2}{2}}} y_i = \sum_{i=1}^n \text{soft max}(-\frac{((x-x_i)w)^2}{2}) y_i$$

高斯核指数部分可以视为注意力评分函数，即 $-\frac{(x-x_i)^2}{2}$ ，选择不同的注意力评分函数会导致不同的注意力汇聚操作。



当查询和键不同长度时，可以使用可加性注意力作为评分函数，给定查询 $q \in R^q$ 和键 $k \in R^k$ ，评分函数为

$$a(q, k) = w_v^T \tanh(W_q q + W_k k)$$

$W_q q$ 可以认为是一个线性层，输入为 q ，权重为 W_q ，禁用偏置项； $W_k k$ 同理；连接后经过 $\tanh$ 激活，再乘以权重 w_v^T 。当它们的长度相同时，使用缩放的“点一

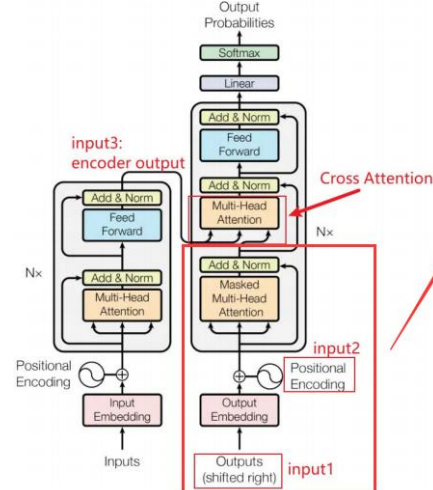
积”注意力评分函数的计算效率更高。

2.2 自注意力机制

在同一序列元素之间计算注意力得分，可以判断某个时刻的数据对另一个时刻有多重要，像 RNN 一样，能够捕获长距离依赖关系。Transformer 模型：完全基于自注意力机制，没有任何卷积层或循环神经网络层。

2.3 交叉注意力机制

用 Decoder（本轨迹）的 Query 查询 Encoder（另一轨迹）的（Key，Value）。



1. Decoder Self-Attention Module:

1. **Query (Q)**: output of the previous layer of the decoder.
2. **Key (K)**: output of the previous layer of the decoder.
3. **Value (V)**: the output of the previous layer of the decoder.

1. Encoder-Decoder Attention Module:

1. **Query (Q)**: output of the decoder's self-attention module.
2. **Key (K)**: encoder's output.
3. **Value (V)**: encoder's output.